TP1 — Cascade, couches et spécificité

Module CSS avancé · Licence 3 · Jour 1 · 10h45 – 11h40

Objectifs pédagogiques

À l'issue de ce TP, vous devez être capable de :

1. **calculer la spécificité** d'un sélecteur (y compris avec `:is()`, `:where()`, `:has()`) et prédire la déclaration retenue en cas de conflit ;
2. **organiser la cascade avec `@layer`** : déclarer un ordre de couches, ranger une bibliothèque tierce dans une couche faible avec `@import … layer()` ;
3. **expliquer** pourquoi l'ordre des couches passe avant la spécificité, et pourquoi un style non rangé dans une couche l'emporte sur tous les styles rangés ;
4. **baisser volontairement la spécificité** avec `:where()` et éviter le piège de `:is()` ;
5. **styler un parent selon son contenu** avec `:has()` (carte avec ou sans visuel, formulaire contenant un champ invalide) ;
6. **utiliser `revert-layer`** pour revenir au style d'une couche précédente ;
7. **écrire un composant en nesting natif** (`&`, `&:hover`, `&.variante`, `@media` imbriquée).

Prérequis

- Cours 1 (cascade, spécificité, `@layer`, mots-clés d'héritage, `:is()` / `:where()` / `:has()`, nesting).
- Savoir ouvrir l'onglet **Éléments → Styles** des DevTools et lire les règles barrées.

Durée et organisation

Durée	55 min (étapes 0 à 6 ; exercice d'approfondissement facultatif)
Organisation	en groupe de trois (trinôme), sur un seul poste. Alternez au clavier à chaque étape : pendant qu'un membre code, les deux autres prédisent le résultat et le vérifient dans les DevTools. Chaque membre doit coder.
Fichiers	`starter/index.html` (ne pas modifier), `starter/vendor.css` (**interdit** de le modifier), `starter/style.css` (le seul fichier que vous modifiez), `starter/labo.html` (étape 0)
Navigateur	Chrome ou Edge récent. Ouvrez `index.html` par double-clic, puis **F12**

Livrable et critères de réussite

- **Livrable** : le fichier `starter/style.css` complété, présenté à l'enseignant en fin de séance (page ouverte dans le navigateur, `style.css` affiché).
- **Critères de réussite** : la liste détaillée figure en fin d'énoncé. Servez-vous-en pour vous auto-évaluer au fil des étapes ; l'enseignant la vérifie en fin de séance. Ce TP relève de l'évaluation formative : il n'est pas noté.

- L'exercice d'approfondissement est **facultatif**.

Contexte de l'exercice

Le site d'un bar à jeux de société, *Le Repaire des Dés*, doit charger tel quel la bibliothèque tierce *MégaKit UI* (`vendor.css`), dont les styles écrasent ceux du site ; un stagiaire a tenté de corriger le problème avec un `!important`. Vous devez rétablir la charte graphique du site **sans `!important`, sans modifier `vendor.css` ni le HTML**, à une exception près : le **bouton de la zone partenaire** conserve l'apparence officielle de MégaKit.

Ouvrez `starter/index.html` : c'est l'état « après MégaKit » (titres magenta en majuscules, police Times, cartes jaunes à bordure pointillée, boutons orange fluo avec ombre noire). Le rendu final attendu est propre, sobre et lisible : police système, titres anthracite, cartes blanches arrondies, boutons ronds vermillon.

Étape 0 — Exercice préparatoire : prédire la couleur (≈ 8 min)

Pour chaque cas, **notez d'abord la couleur sur papier**, en écrivant la raison (spécificité ? couche ? hors couche ? héritage ?). Ensuite seulement, collez le CSS dans la balise `<style id="cas">` de `starter/labo.html` et vérifiez. Chaque prédiction doit être justifiée par écrit ; la correction est faite collectivement avec l'enseignant à la fin de l'étape.

HTML commun aux 5 cas :

```
<div class="boite">
  <p id="intro" class="texte">Qui suis-je ?</p>
</div>
```

Cas 1

```
#intro { color: goldenrod; }
.boite .texte.texte { color: royalblue; }
```

Cas 2

```
@layer base, theme;
@layer theme { p { color: seagreen; } }
@layer base  { #intro { color: crimson; } }
```

Cas 3

```
@layer composants { #intro.texte { color: purple; } }
p { color: darkorange; }
```

Cas 4

```
:is(.texte, #intro) { color: teal; }
.boite p.texte { color: navy; }
```

Cas 5

```
@layer base, composants;
@layer base { p { color: crimson; } }
@layer composants { .texte { color: revert-layer; } }
.boite { color: hotpink; }
```

Indice (si vous bloquez sur plus de 2 cas)

L'ordre de comparaison est le suivant : **origine et importance**, puis **couche**, puis **spécificité**, puis **ordre d'apparition**. La spécificité n'est comparée que si les deux règles sont dans la même couche (ou toutes les deux hors couche). Une valeur **héritée** du parent n'entre même pas dans la comparaison : elle ne s'applique que si personne ne cible l'élément.

Étape 1 — Diagnostiquer, puis enfermer MégaKit dans une couche (≈ 7 min)

1. Dans les DevTools, inspectez le titre **« Les jeux de la semaine »**. Notez le sélecteur de MégaKit qui gagne et **sa spécificité**. Expliquez pourquoi votre règle `h1, h2, h3 { … }` est barrée.
2. En tête de `style.css` (bloc `TODO étape 1`), **déclarez l'ordre des couches** : `reset, vendor, base, layout, components, utilities`.
3. Modifiez l'`@import` pour ranger **tout** `vendor.css` dans la couche `vendor`.
4. Rechargez la page : que se passe-t-il ? Pourquoi est-ce que **tout votre CSS gagne déjà**, alors qu'il n'est encore dans aucune couche ?
5. Supprimez le `!important` du stagiaire (`.nav__lien`) et vérifiez que la navigation reste blanche.

Indice 1 — la syntaxe

```
@layer a, b, c;                       /* déclaration de l'ordre : c gagne */
@import url("fichier.css") layer(a);  /* tout le fichier va dans la couche a */
```

`@import` doit précéder toutes les autres règles, **sauf** `@charset` et les déclarations `@layer` : vous pouvez donc mettre la déclaration d'ordre avant l'import.

Indice 2 — pourquoi tout gagne déjà ?

Les styles **hors couche** forment une couche implicite placée **après toutes les autres** : ils battent donc tout ce qui est rangé dans une couche, quelle que soit la spécificité.

Étape 2 — Ranger votre propre CSS dans les couches (≈ 10 min)

1. Enveloppez chaque grande section de `style.css` dans sa couche : `@layer reset { … }`, `@layer base { … }`, `@layer layout { … }`, `@layer components { … }`, `@layer utilities { … }` (repérez les commentaires `TODO étape 2`).
2. Dans le **reset**, remplacez les listes de sélecteurs par `:where(…)` pour mettre leur spécificité à 0.

3. Rechargez. Deux problèmes apparaissent ; résolvez-les **sans** `!important` :
   - **Problème A** : la police reste en **Times/Georgia** partout, alors que `body { font-family: … }` est dans `base`, une couche plus forte que `vendor`.
   - **Problème B** : les boutons sont ronds et vermillon… mais gardent **une ombre noire, une bordure noire et des MAJUSCULES**. Les puces carrées de MégaKit sont toujours là, dans la navigation et devant les chiffres du hero.

> **Indice — problème A (police)**
>
> Regardez **sur quel élément** MégaKit déclare `font-family`. Une valeur **héritée** perd toujours contre une déclaration qui cible directement l'élément, même si cette déclaration vient d'une couche faible. Le HTML vous offre une classe sur ce même élément…

> **Indice — problème B (ombre, majuscules, puces)**
>
> Les couches ne **suppriment** pas les règles de `vendor` : elles gagnent seulement les **conflits**. Toute propriété que vous ne redéclarez pas (`box-shadow`, `text-transform`, `border`, `font`…) continue de venir de MégaKit. Pour les puces : MégaKit met `list-style` sur les `<li>`, pas sur les `<ul>`. Votre règle doit donc viser les `<li>` (même avec une spécificité de 0 grâce à `:where()`, elle gagnera : elle est dans une couche plus forte).

Étape 3 — Le piège de `:is()` (≈ 5 min)

Le titre de la carte **« La Forêt des Lucioles »** (variante `.card--coup-de-coeur`) devrait être vermillon. Il ne l'est pas.

1. Calculez la spécificité de `:is(#jeux, .agenda) .card-title` et de `.card--coup-de-coeur .card-title`.
2. Corrigez en **changeant un seul mot**.

> **Indice**
>
> `:is()` prend la spécificité de **son argument le plus fort**, même si l'élément a été trouvé grâce à un argument plus faible. Il existe une pseudo-classe identique à `:is()` mais dont la spécificité vaut toujours **0**.

Étape 4 — Le composant carte en nesting natif (≈ 8 min)

Réécrivez les 7 règles `.card…` du starter en **une seule règle** `.card { … }` utilisant le nesting natif :

- `&:hover` (et ajoutez-y une ombre plus marquée) ;
- `& .card__corps`, `& .card__meta`, `& .card__visuel` ;
- `&.card--coup-de-coeur { … & .badge { … } }`.

Pour aller plus loin (facultatif) : faites de même pour `.nav__lien` et `.btn`, et **imbriquez une media query** dans `.hero .hero__inner`.

Indice — & avec ou sans espace ?

Écrit dans `.card { }`	Équivaut à	Sens
`&:hover`	`.card:hover`	le même élément, survolé
`& .card__corps`	`.card .card__corps`	un **descendant**
`&.card--coup-de-coeur`	`.card.card--coup-de-coeur`	le **même** élément qui a aussi la classe
`@media (width >= 56rem) { … }`	la même règle dans une media query	les déclarations s'appliquent à `.card`

La spécificité est la même que dans la version « à plat ». Vérifiez dans les DevTools que le rendu n'a pas bougé.

Étape 5 — `:has()` : styler le parent selon son contenu (≈ 10 min)

5a. Cartes sans visuel. Deux cartes n'ont pas encore d'illustration. Sans ajouter de classe dans le HTML, donnez aux cartes **qui ne contiennent pas** de `.card__visuel` un bandeau décoratif en haut (pseudo-élément `::before`, `aspect-ratio: 16 / 9`, dégradé rayé sarcelle) pour qu'elles s'alignent visuellement sur les autres.

5b. Formulaire.

1. Ajoutez une **étoile** `*` après le label de chaque `.champ` qui contient un champ obligatoire.
2. Un `.champ` qui contient un champ invalide passe en rouge : label, bordure du champ, texte d'aide.
3. Le message `.form__erreur` est **masqué** par défaut et ne s'affiche **que si le formulaire contient** au moins un champ invalide ; la bordure du formulaire devient rouge.

Testez : saisissez `pas-un-email` dans E-mail et `40` dans Nombre de joueurs, puis cliquez ailleurs.

Indice 1 — sélecteurs utiles

`.parent:has(.enfant)`, `.parent:not(:has(.enfant))`, `:required`, `:user-invalid`.

Indice 2 — pourquoi :user-invalid et pas :invalid ?

`:invalid` est vrai **dès le chargement** (un champ obligatoire vide est invalide) : le formulaire serait rouge avant même que l'utilisateur ait tapé quoi que ce soit. `:user-invalid` ne s'active qu'**après une interaction** (saisie puis sortie du champ, ou tentative d'envoi).

Étape 6 — `revert-layer` : restaurer le style de la bibliothèque (≈ 5 min)

Le contrat impose l'apparence **officielle MégaKit** pour le bouton de la zone partenaire (`.partenaire .btn`). En **une seule déclaration**, annulez tout ce que la couche `components` a décidé pour ce bouton.

Expliquez oralement pourquoi c'est bien le style de `vendor` qui réapparaît, et pas celui de `base`.

Indice

`revert-layer` fait comme si la couche courante n'avait rien déclaré pour cette propriété : la cascade repart des couches **précédentes**. Pour toutes les propriétés d'un coup, utilisez la propriété raccourcie `all`. Vérifiez la spécificité de votre règle par rapport à `.btn`, dans la même couche.

Exercice d'approfondissement (facultatif) — Filtrer les jeux sans JavaScript

Au-dessus des cartes se trouvent 4 boutons radio (`#f-tous`, `#f-coop`, `#f-ambiance`, `#f-strategie`).

1. Transformez-les en **puces cliquables** : radio invisible mais toujours focusable au clavier, puce « pleine » quand elle contient le radio coché, contour visible au focus clavier.
2. Quand « Coopératif » est coché, **seules les cartes** `data-type="coop"` restent visibles (même chose pour les autres filtres). « Tous » affiche tout.

Contraintes : zéro JavaScript, zéro `!important`, et le tout dans la couche `components`.

Indice

La section `.jeux` contient à la fois les radios **et** les cartes : `.jeux:has(#f-coop:checked) .card:not([data-type="coop"])`. Pour la puce : `.filtres__puce:has(:checked)` et `.filtres__puce:has(:focus-visible)`.

Critères de réussite

Auto-évaluez-vous avec cette liste ; l'enseignant la vérifie en une minute, page ouverte et `style.css` affiché :

- ☐ `style.css` commence par `@layer reset, vendor, base, layout, components, utilities;` suivi de `@import url("vendor.css") layer(vendor);`.
- ☐ **Aucun** `!important` dans `style.css` (Ctrl+F) et `vendor.css` intact.
- ☐ Police système partout (plus de Times/Georgia), titres anthracite sans majuscules forcées, boutons sans ombre noire, plus de puces carrées.
- ☐ Le titre « La Forêt des Lucioles » est vermillon, grâce à `:where()` (et non par une hausse de spécificité).
- ☐ `.card` est écrite en **un seul bloc imbriqué**.
- ☐ Les deux cartes sans illustration ont un bandeau rayé, obtenu avec `:has()`.
- ☐ Au chargement, pas de message d'erreur. Après saisie d'un e-mail invalide, le champ et le formulaire passent en rouge et le message apparaît. Les labels obligatoires portent une `*`.
- ☐ Le bouton « Découvrir MégaKit » a le style orange carré de MégaKit, alors que tous les autres boutons sont ronds (via `revert-layer`).

Exercice d'approfondissement (facultatif) : réussi si les 4 filtres fonctionnent à la souris **et** au clavier (Tab puis flèches), sans JavaScript.

Questions de réflexion (à l'oral, lors de la vérification par l'enseignant)

1. La règle `:where(h1, h2, h3)` a une spécificité de **(0,0,0)** et bat pourtant `#app h1` **(1,0,1)**. Pourquoi ? Que se passerait-il si un collègue ajoutait un `h2 { color: red; }` **hors de toute couche** à la fin du fichier ?
2. Imaginons que MégaKit contienne `#app .btn { text-transform: uppercase !important; }`. Un `!important` dans votre couche `components` suffirait-il à le battre ? Quelle couche « gagne » pour les déclarations `!important`, et qu'en déduisez-vous sur l'usage de `!important` dans une architecture en couches ?
3. Vous écrivez un **reset** ou une **bibliothèque** destinée à d'autres développeurs : préférez-vous `:is()` ou `:where()` ? Et dans quel cas `:is()` reste-t-il le bon choix ?